



Editorial

Tasks, test data and solutions were prepared by: Ivan Janjić, Sofija Velkovska, and Jakov Celin.

Implementation examples are given in attached source code files.

Task Kvadrat i Kub

Prepared by: Sofija Velkovska

Required knowledge: input and output statements

First, we read the number x . The square of x is by definition equal to $x \cdot x$, so we print $x \cdot x$. The cube of x equals $x \cdot x \cdot x$ — we print $x \cdot x \cdot x$ on the second line.

Source code (written in Python 3):

```
x = int(input())

print(x * x)
print(x * x * x)
```

Task Lozinka

Prepared by: Sofija Velkovska

Required knowledge: conditional statement (if)

First, note that n is at most 19 (see the “Input” section in the problem statement).

Furthermore, note that the first 9 natural numbers consisting of only a single distinct digit are 1, 2, 3, 4, 5, 6, 7, 8, and 9. Therefore, if n is between 1 and 9, Martin’s password will be exactly the number n .

Next, we consider two-digit numbers consisting of only one distinct digit — these are 11, 22, 33, 44, 55, 66, 77, 88, and 99. Notice that these numbers can be obtained as 11 multiplied by some number ($11 = 1 \cdot 11, 22 = 2 \cdot 11, \dots, 99 = 9 \cdot 11$). The 10th such number is $1 \cdot 11$, the 11th is $2 \cdot 11$, ..., the 18th is $9 \cdot 11$. In one formula, this can be expressed as $(n - 9) \cdot 11$.

The remaining case is $n = 19$. After 99, the next number consisting of only one distinct digit is 111 — we print it, since that is Martin’s password.

Source code (written in Python 3):

```
n = int(input())

if n <= 9:
    print(n)
if n > 9 and n <= 18:
    print((n - 9) * 11)
if n == 19:
    print(111)
```

Task Igra

Prepared by: Sofija Velkovska

Required knowledge: conditional statement (if)

If $a = b$, we clearly do not need to perform any operations since a is already equal to b . Therefore, 0 operations are needed to transform a into b .



If $a < b$, we can increase a by the natural number $b - a$. We will get $a = b$ after one operation.

If $a > b$, we can transform a into b using only one operation if there exists a natural number k such that k exactly divides a and $\frac{a}{k} = b$, i.e. $a = b \cdot k$. Such a number exists only if b exactly divides a — we can check this using the modulo operator.

If none of these conditions hold (i.e. $a > b$ and b does not divide a), then we need two operations to transform a into b . Every number divides itself, so we can divide a by a to get 1, and then increase 1 by $b - 1$ to obtain b . We have thus used two operations to transform a into b .

Source code (written in Python 3):

```
a, b = map(int, input().split())

if a == b:
    print(0)
elif a < b or a % b == 0:
    print(1)
else:
    print(2)
```

Task Čokolada

Prepared by: Sofija Velkovska

Required knowledge: matrices, loop (for), conditional statement (if)

Suppose two adjacent squares in the same row differ in color, i.e. one is black and the other is white. Let these be the j -th and $(j + 1)$ -th squares in the i -th row. Luka will certainly have to make a cut between the j -th and $(j + 1)$ -th columns. Otherwise, these squares would end up in the same piece of chocolate at the end of the cutting process, which is not allowed since one is black and the other is white.

If there exists another row, denote it k , in which the j -th and $(j + 1)$ -th squares also differ in color, they will be separated by the cut we concluded must be made (the cut separating the j -th and $(j + 1)$ -th squares in the i -th row). Therefore, Luka can resolve all adjacent differently-colored squares in the j -th and $(j + 1)$ -th columns with a single cut.

More precisely, if the j -th and $(j + 1)$ -th squares in every row are pairwise the same color, Luka does not need to make a cut between the j -th and $(j + 1)$ -th columns. If at least one row has squares of differing colors, a cut is necessary.

The same applies to horizontal cuts Luka can make (in that case we examine the j -th square in the i -th and $(i + 1)$ -th rows). It is easy to see that after all cuts, no two squares of different colors will remain in the same rectangular piece.

To compute the minimum number of cuts needed to separate the black and white squares, we must check the described condition for all pairs of adjacent columns and rows and track the total number of required cuts.

Task Džeparac

Prepared by: Jakov Celin

Required knowledge: dynamic programming, fast exponentiation

The problem is solved by finding a formula in various ways (formal proofs, intuition, ...), so we will describe one approach.

Let $r = N - k$ denote the amount that mother Antonija distributes to her sons. Since each son must receive the same total amount, r must be divisible by 2. Let $m = r/2$ be the total amount each son



receives. The distribution of amount m over days corresponds to a *composition* of m , i.e. a choice of an ordered sequence of positive integers $(x_1, x_2, \dots, x_d)$ whose sum equals exactly m . The number of such compositions for a fixed $m \geq 1$ equals 2^{m-1} , since each of the $m - 1$ gaps between unit parts of the amount can either be within the same day or on the boundary between days.

The total number of schedules is therefore

$$1 + \sum_{\substack{r \in \{2, 4, \dots\} \\ r \leq N}} 2^{r/2-1},$$

where the first term corresponds to the case $k = N$, $d = 0$, and the sum runs over all even values of r . Substituting $j = r/2$, where j ranges from 1 to $\lfloor N/2 \rfloor$, we get

$$1 + \sum_{j=1}^{\lfloor N/2 \rfloor} 2^{j-1} = 1 + (2^{\lfloor N/2 \rfloor} - 1) = 2^{\lfloor N/2 \rfloor},$$

where we used the geometric series formula $\sum_{j=1}^t 2^{j-1} = 2^t - 1$.

The final answer is therefore $2^{\lfloor N/2 \rfloor} \bmod (10^9 + 7)$. Since $N \leq 10^{18}$, the exponent $\lfloor N/2 \rfloor \leq 5 \cdot 10^{17}$, so the answer must be computed using fast exponentiation in $O(\log N)$ time.

Task Prepisivanje

Prepared by: Jakov Celin

Required knowledge: maximum bipartite matching, maximum independent set in a bipartite graph, König's theorem

The problem condition states that no student may have a neighbor (in four directions) who is also a student. We are therefore looking for the largest set of seats where students (well-behaved or mischievous) can sit, such that no two chosen seats are adjacent — which is exactly the definition of the *maximum independent set* of a graph.

We construct the graph as follows. The vertices of the graph are all free seats (cells with value 0) that are **not** adjacent to well-behaved students (cells with value 2). An edge between two vertices exists if and only if the corresponding cells share a side. Cells with value 1 are ignored entirely — no one can sit in them and they do not affect the copying condition. The number of well-behaved students is fixed and all of them must be included in the solution.

The adjacency graph of the grid cells is naturally *bipartite*: if we color the cells in a checkerboard pattern (black-white according to the parity of $i + j$), then every edge in the grid connects a cell of one color to a cell of the other. We treat cells with well-behaved students as pre-selected vertices and remove all adjacent empty cells (they cannot be selected). On the remaining graph (consisting only of the remaining free cells) we find the maximum independent set.

The maximum independent set in a bipartite graph can be determined by combining two classical results. The first, König's theorem, states that the size of the *minimum vertex cover* equals the size of the *maximum matching*. The second is that the independent set is the complement of the vertex cover, so

$$|\text{maximum independent set}| = |V| - |\text{minimum vertex cover}| = |V| - |\text{maximum matching}|,$$

where $|V|$ is the number of vertices (remaining free cells). The maximum bipartite matching is found using the standard algorithm which, on a graph with $|V| \leq n \cdot m \leq 6400$ vertices and $O(nm)$ edges, runs within the time limit with complexity $O(V \cdot B)$ where $B \leq 4 \cdot nm$ is the number of edges in the bipartite graph.

The final answer is the number of well-behaved students (all cells with value 2) increased by the size of the maximum independent set on the graph of remaining free cells.



Task Skijanje

Prepared by: Jakov Celin

Required knowledge: Li-Chao tree, monotone deque

Before reading this solution, it is recommended to carefully study the following blog posts, which describe in detail the key techniques used in the solution:

- [Li Chao tree and Convex Hull Trick](#)
- [Monotone deque](#)

Every run in the tree leads to exactly one node $v > 1$, so a descent of at most k consecutive runs is uniquely described by choosing the *first* node f and the *last* node l of the descent, where f is an ancestor of l at a distance between 1 and k (distance 1 corresponds to a descent of a single run). Let us also introduce the prefix sums of speeds along the path from the root: let $S(v) = \sum_{u \text{ proper ancestor of } v} b_u$, with the convention $S(\text{root}) = 0$. The sum of speeds of all runs on the descent from f to l is then

$$\sum_{\text{runs on descent}} b_i = S(l) + b_l - S(f).$$

The excitement of the descent can be written as

$$\begin{aligned} H(f, l) &= z_l \cdot (z_l + S(l) + b_l - S(f)) + z_f^2 \\ &= z_l^2 + z_l \cdot (S(l) + b_l) - z_l \cdot S(f) + z_f^2. \end{aligned}$$

For a fixed node l , the first two components are constants that do not depend on f , so maximizing $H(f, l)$ over all allowed ancestors f reduces to maximizing the expression

$$-z_l \cdot S(f) + z_f^2.$$

We associate each node v with a vector $\mathbf{p}(v) = (S(v), -z_v^2)$, and define the query vector for l as $\mathbf{q}(l) = (z_l, 1)$. The value we are maximizing then equals (with a sign change)

$$-\mathbf{p}(f) \cdot \mathbf{q}(l) = -(S(f) \cdot z_l + (-z_f^2) \cdot 1) = -z_l \cdot S(f) + z_f^2,$$

which reduces the problem to: for each node l , find the minimum dot product $\mathbf{p}(f) \cdot \mathbf{q}(l)$ over the set of ancestors f of l located between 0 and $k - 1$ levels away, and negate the result.

Note that for a fixed l the query vector has the form $(z_l, 1)$, so

$$\mathbf{p}(f) \cdot \mathbf{q}(l) = S(f) \cdot z_l + (-z_f^2) \cdot 1.$$

Each vector $\mathbf{p}(f)$ can be interpreted as a line $y = S(f) \cdot x + (-z_f^2)$, and a query at $x = z_l$ asks for the minimum value of that line. This is a classic problem that a *Li-Chao tree* solves, answering such queries in $O(\log M)$ time, where M is the range of values z_v , i.e. $M = 10^5$.

The set of allowed ancestors changes as the DFS traversal visits nodes: upon entering node l , l itself joins the set (as a potential f for descendants), and the node that is $k + 1$ levels above l leaves. Upon exiting l , that node rejoins the set. Therefore, the set of ancestors we maintain must support insertion and deletion at both ends, where each operation must update the Li-Chao tree. Since the Li-Chao tree **does not** support deletions, a monotone deque with Li-Chao trees supporting rollback of the **last** insertion is used. Each element is added and removed $O(1)$ times throughout the entire DFS, and each insertion and deletion has time complexity $O(\log M)$ due to the Li-Chao tree. The global answer is obtained by taking the maximum over all nodes l of the values

$$\text{ans} \leftarrow \max \left(z_l^2 + z_l \cdot (S(l) + b_l) + \max_f (-z_l \cdot S(f) + z_f^2) \right).$$

The total time complexity of the algorithm is $O(N \log M)$, and the space complexity is $O(N + M)$.



Task Učionica

Prepared by: Sofija Velkovska

Required knowledge: binary search

Construct the matrix $maxcol$ such that $maxcol_{i,j}$ equals the maximum of all $a_{l,j}$ for which $l \leq i$. A person of height h can sit in seat $(i+1, j)$ if the seat is empty ($a_{i+1,j} = 0$) and $h > maxcol_{i,j}$. In the first row, a person can sit in seat $(1, j)$ if $a_{1,j} = 0$.

For the first subtask, we iterate over all pairs of adjacent seats (i, j) and $(i, j+1)$ and check whether either of the following two possibilities satisfies the conditions: the friend of height h_1 sits at (i, j) and the friend of height h_2 at $(i, j+1)$, or vice versa.

For the second subtask, we consider all consecutive groups of k seats. Assume all those seats are empty. It is optimal to place taller friends in columns where the tallest student in front of them is taller, and shorter friends behind shorter students. Therefore, for seats $(i, j), (i, j+1), \dots, (i, j+k-1)$, we sort the values $maxcol_{i,j}, maxcol_{i,j+1}, \dots, maxcol_{i,j+k-1}$ and assign seats to friends according to this order: the tallest friend gets the column with the highest value, the second tallest gets the next, and so on. We then check whether each friend is taller than the tallest person in front of them. This solution runs in $O(nmk \log k)$ time.

Now let us ignore seat occupancy and focus only on the height constraints. If the seats starting at (i, j) form a valid group, then the seats starting at (l, j) for any $1 \leq l \leq i$ also form a valid group. Therefore, for a fixed j , there exists a threshold t such that rows 1 through t are valid, while rows $t+1$ through n are not. We can find t using binary search. We then check which of those valid seat groups have all k seats empty. Using a sliding window technique, this can be done in $O(1)$ per row, giving a complexity of $O(mk \log n \log k)$.

For the final subtask, we remove the $\log k$ factor. Consider the k values $maxcol$: $ms_1, ms_2, \dots, ms_k$ and the friends' heights $h_1, h_2, \dots, h_k$. We need to check whether, after sorting, each ms value is less than the corresponding h value. This holds if there is one ms value smaller than all heights, another smaller than at least $k-1$ heights, a third smaller than at least $k-2$ heights, and so on.

Before the main algorithm, for each (i, j) we precompute how many of the friends' heights are greater than $maxcol_{i,j}$. This can be done in $O(n^2)$ time. With these values known, we can easily check whether there is at least one value greater than or equal to k , another greater than or equal to $k-1$, and so on, using an auxiliary array. Since this check takes $O(k)$ time, the final time complexity is $O(mk \log n)$.